Income Tax ChatBot: A Retrieval-Augmented Generation System

SMAI Assignment 3 (Topic 10.4)

Aikya Oruganti - 2023102062
Pragnya Tatiparthi - 2023102067

1 Introduction

Indian taxpayers face significant complexity navigating Income Tax documents, validation rules, and filing deadlines. Official CBDT (Central Board of Direct Taxes) PDFs contain authoritative but dense information across multiple ITR forms (ITR-1 through ITR-7).

This project addresses the problem: *How can we make tax information accessible through natural language queries while maintaining accuracy and traceability?*

1.1 Problem Statement

Traditional approaches fail because:

1. Keyword search ignores semantic similarity (e.g., "eligibility" and "qualification" are synonyms)
2. LLMs alone hallucinate tax facts without source grounding
3. Some users need bilingual support (English/Hindi) for Indian taxpayers
4. Citation trails are legally required for tax advice

1.2 Our Approach

We adopt a hybrid retrieval-augmented generation (RAG) architecture:

1. **Offline Phase**: Chunk and embed official PDFs into a vector database (ChromaDB)
2. **Online Phase**: Embed user queries, retrieve top-k relevant chunks, and augment LLM prompts with grounded context
3. **Fallback Mechanism**: Web crawling for recently updated information when PDFs are insufficient

1.3 Contributions

- **Empirical Ablation Study**: Systematic comparison of chunk sizes (200, 400, 800 chars) with 10 standardized tax queries, revealing semantic density trade-offs
- **Production Architecture**: Modular design with Streamlit UI, bilingual support, and source attribution
- **Practical Insights**: Identification of design tensions (quality vs. storage, determinism vs. coverage) and their resolution

2 Data

2.1 Source Documents

The system processes official CBDT e-filing validation rules and guidelines. 7 ITR Validation Rules Files and other FAQ documents.

2.2 Document Characteristics

- **Domain Specificity**: 100% domain-specific (Indian Income Tax) with minimal general text contamination
- **Structure**: Dense and hierarchical with sections, subsections. PDF extraction via PyMuPDF preserves approx structure via form-feed characters
- **Language**: Docs are in English.

2.3 Test Question Set

For ablation studies, we define 10 representative queries:

1. "Who is eligible to file ITR-1?"
2. "What is the last date to file ITR?"
3. "How is TDS on salary calculated?"
4. "What is the 87A rebate?"
5. "Can a non-resident file ITR-2?"
6. "What are the sections under Chapter VI-A deductions?"
7. "What is the penalty for late filing?"
8. "How to report capital gains?"
9. "What is the standard deduction for salaried employees?"
10. "Is it mandatory to link PAN with Aadhaar?"

These cover eligibility, deadlines, calculations, deductions, penalties, and compliance representing the typical taxonomy of user questions.

3 Method

3.1 Architecture Overview

The system comprises three layers:

3.1.1 Layer 1: Offline Indexing (build_index.py)

```
# Extract pages from PDFs
pages = pypdf.PdfReader(path).pages
# Fixed-size overlapping chunks
chunks = text[start:start+SIZE]
# Embed with all-MiniLM-L6-v2 (22M params)
embeddings = model.encode(chunks)
# Store in ChromaDB (persistent)
collection.add(documents, embeddings, ids, metadatas)
```

Listing 1: Text extraction and chunking pipeline

- **PDF Extraction**: PyMuPDF for robust text extraction
- **Chunking Strategy**: Fixed-size character chunks with 50-char overlap (tuned via ablation)
- **Embedding Model**: `sentence-transformers/all-MiniLM-L6-v2` (384-dim vectors)
- **Storage**: ChromaDB persistent client for reproducibility and offline availability

3.1.2 Layer 2: Online Retrieval (app.py retrieval functions)

```
q_embedding = model.encode(question)
results = collection.query(
    query_embeddings=[q_embedding],
    n_results=TOP_K,
    include=["documents", "metadatas", "distances"]
)
score = max(0.0, 1.0 - distance / 2.0)  # Normalize L2 distance
```

Listing 2: Query-time retrieval and ranking

- **Semantic Search**: Cosine similarity on 384-dim embeddings
- **Relevance Scoring**: Normalize L2 distance to $[0, 1]$ via $\text{score} = \max(0, 1 - d/2)$
- **Top-k Retrieval**: $k = 3$ (tuned heuristically. Larger k adds noise, smaller k reduces coverage)

3.1.3 Layer 3: LLM Augmentation (Groq API)

```
prompt = f"""You␣are␣a␣helpful␣Income␣Tax␣assistant␣for␣India.
PDF␣CONTEXT:
{context_text}␣␣#␣Top-3␣chunks␣with␣source␣attribution
QUESTION:␣{question}
ANSWER␣(end␣with␣Sources:):"""
# Call Groq llama-3.3-70b-versatile, temp=0.4, max_tokens=1000
```

Listing 3: Prompt construction with context and memory

- **Model**: Groq's `llama-3.3-70b-versatile` (faster inference via tensor parallelism)
- **Temperature**: 0.4 (low to minimize hallucination and balances determinism with fluency)
- **Context Window**: Include retrieval confidence, memory of last 3 turns, web fallback results
- **Fallback Strategy**: If top score < 0.5 or < 2 chunks retrieved, crawl web for PDFs

3.2 Web Crawling Implementation

Architecture: The system uses a two-stage web crawling pipeline with PDF discovery and extraction layers.

PDF Discovery: The `crawl_website_for_pdfs()` function searches predefined tax websites (incometax.gov.in) using BeautifulSoup HTML parser. It identifies direct PDF links and performs recursive crawling on pages containing tax-related keywords: *itr, tax, return, filing, deduction.*

Content Extraction: The `download_and_process_pdf()` function downloads PDFs via HTTP requests and extracts text using PyMuPDF, returning up to 1000 characters for context.

Fallback Mechanism: Web crawling activates when vector database retrieval returns fewer than 2 relevant chunks, ensuring answer coverage through online sources. It crawls maximum 2 URLs and processes up to 2 PDFs per URL.

Integration: Extracted web content is merged into the LLM prompt alongside local PDF context. The system flags web-sourced answers to users and maintains proper source attribution for all citations.

3.3 Bilingual Design

The UI supports English and Hindi via a simple toggle:

- **Interface**: Streamlit's i18n-like pattern with TEXTS dictionary
- **Limitation**: Embeddings are English-only. Hindi queries are handled by transliteration.
- **Future Work**: Fine-tune embeddings on Hindi tax terms or use multilingual models

4 Results

4.1 Ablation Study: Impact of Chunk Size

We systematically evaluated three chunk sizes on the same 10 test questions using the same embedding model and retrieval metric.

4.1.1 Setup

- **Independent Variable**: Chunk size in characters (200, 400, 800)
- **Dependent Variable**: Average retrieval score (relevance of top-1 result)
- **Fixed Variables**: Test questions, overlap (50 chars), embedding model, corpus, scoring formula
- **Metric**: $\text{Score}_i = \max(0, 1 - d_i/2)$ where d_i is L2 distance to top result for question i

4.1.2 Findings

Chunk Size (chars)	Total Chunks	Avg. Score	Score Density
200	6,220	0.6762	0.000109
400	2,667	0.6472	0.000243
800	1,247	0.5751	0.000461

Table 1: Ablation results: chunk size impact on retrieval quality

Key Observation: Smaller chunks achieve **17.6% higher** average score than 800-char chunks (0.6762 vs. 0.5751).

4.1.3 Analysis: Why Small Chunks Win

Semantic Density Hypothesis Tax documents are *semantically dense*. A single 800-char chunk may contain:

- ITR-1 eligibility rules (one concept)
- Section 80C deduction limits (second concept)
- TDS rates (third concept)

When a user asks "Who is eligible for ITR-1?", the embedding of that query is pulled toward *all three concepts* in the 800-char chunk. The embedding becomes **less pure**, reducing cosine similarity to the actual query intent.

Conversely, a 200-char chunk focuses on a single concept, producing a sharper embedding that better matches the query.

Optimal Trade-off: 400 Chars Despite 200-char chunks scoring highest, we choose **400 chars as production standard** because:

1. Score difference (0.6762 vs. 0.6472) is only **4.5%**, within noise margins
2. 400-char chunks reduce storage/compute by 2.3× compared to 200-char
3. 400-char chunks preserve **sufficient context** for the LLM to generate coherent answers
4. Empirically, 200-char chunks produce *fragmented answers* lacking narrative flow

4.2 Qualitative Performance

We manually tested representative queries:

1. **Query**: "What is the 87A rebate?"
   Result: Retrieved correct section with tax savings formula; correctly cited ITR-1 document and page.
2. **Query**: "Can a non-resident file ITR-2?"
   Result: Retrieved eligibility rules, correctly stated residency requirements.
3. **Query**: "How is TDS on salary calculated?"
   Result: Fallback to web crawl (not in PDFs).Handled via disclaimer.

Observation: System correctly distinguishes between in-PDF and out-of-scope queries.

5 Limitations

5.1 Technical Limitations

1. **Embedding Model Mismatch**: `all-MiniLM-L6-v2` is trained on general English; not fine-tuned on tax terminology. Terms like "ITR," "TDS," and "Section 80C" lack domain-specific semantic positioning.
2. **Hallucination Risk**: LLM may invent plausible-sounding tax advice outside the retrieved context. We mitigate via low temperature (0.4) and explicit instructions to cite sources, but cannot eliminate entirely.
3. **Bilingual Bottleneck**: English embeddings do not capture Hindi tax terminology nuances. A Hindi query is embedded in English space, losing semantic precision.
4. **Overlap Hardcoding**: Chunk overlap fixed at 50 chars, no adaptive overlap based on text boundary semantics (e.g., sentence or paragraph breaks).

5.2 Data Limitations

1. **Static Corpus**: PDFs reflect Assessment Year 2025-26 rules. Tax laws change annually, requires manual corpus updates.
2. **Limited Test Set**: 10 queries for ablation, may not represent long-tail user questions (e.g., edge cases in specific sections).

5.3 Architectural Limitations

1. **No Query Rewriting**: User queries are directly embedded without reformulation. "Is ITR-1 filing mandatory?" and "Who must file ITR-1?" may have slightly different embeddings despite similar intent.
2. **Fixed Top-k**: $k = 3$ chunks retrieved regardless of query complexity. Ambiguous queries may benefit from $k = 5$; narrow queries may waste context with $k = 3$.
3. **Web Fallback Validation**: Crawled web content is usually unvalidated. **used it very minimally and we've used only official websites as fallback websites**
4. **No User Feedback Loop**: System does not learn from user corrections or ratings, embeddings remain frozen.

6 Discussion

6.1 Counter-Intuitive Finding

The **17.6% quality boost from smaller chunks** contradicts common advice in RAG systems (e.g., "use 1000-token chunks for context"). This reflects the unique properties of tax documents: dense technical content where semantic concepts are tightly packed.

Generalizability Question: Does this hold for other domains?

- **Likely Yes**: Legal documents, medical protocols, technical specifications (similarly dense)
- **Likely No**: Narrative text (blogs, novels, customer support), where larger chunks preserve narrative coherence

6.2 Design Tensions

1. **Quality vs. Storage**: Smaller chunks improve retrieval quality by 17.6% but increase storage by 5×. Production choice (400 chars) is a compromise.
2. **Determinism vs. Coverage**: Low temperature (0.4) reduces hallucination but may produce generic answers. High temperature (0.7) is more creative but less trustworthy for legal tax advice.
3. **Speed vs. Accuracy**: Retrieving 3 chunks is 10× faster than retrieving 30 chunks, but may miss relevant information. We use adaptive threshold (retrieve more if confidence low) as compromise.

6.3 Why This Matters

Tax advice carries legal consequences. A chatbot that misidentifies ITR eligibility could lead a taxpayer to file the wrong form, resulting in penalties. This design prioritizes:

1. **Traceability**: Every answer cites source PDF and page number
2. **Grounding**: Context is retrieved from official documents, not hallucinated
3. **Transparency**: Confidence scores and fallback signals inform users when the system is uncertain

7 Implementation Details

7.1 Technology Stack

Component	Technology
PDF Extraction	PyMuPDF (fitz)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Vector Database	ChromaDB (persistent client)
LLM Inference	Groq (llama-3.3-70b-versatile)
Web Crawling	BeautifulSoup + requests
UI Framework	Streamlit
Language	Python 3.13

Table 2: Technology choices and rationale

7.2 Chunking Algorithm

```
def chunk_text(text, size=400, overlap=50):
    step = size - overlap
    chunks = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start+size])
        start += step
```

```
    return chunks
```

Listing 4: Fixed-size overlapping chunking

Rationale: Character-based (not token-based) chunking avoids tokenizer dependency and ensures reproducibility across systems.

7.3 Scoring Formula

$$\text{Score}(q, d) = \max\left(0, 1 - \frac{d(q, d)}{2}\right)$$

where $d(q, d)$ is L2 distance between query and document embeddings. Normalized to $[0, 1]$, the formula assigns:

- Score $= 1.0$ when $d = 0$ (perfect match, rare)
- Score $= 0.5$ when $d = 1$ (typical retrieval quality)
- Score $= 0$ when $d \geq 2$ (poor match)

8 Future Work

1. **Domain-Specific Embeddings**: Fine-tune `all-MiniLM-L6-v2` on tax document pairs to improve semantic grounding.
2. **Multilingual Support**: Adopt multilingual embedding models (e.g., `multilingual-e5-large`) to natively support Hindi queries.
3. **Query Rewriting**: Add an intermediate LLM pass to reformulate user queries into normalized forms before retrieval (e.g., standardize abbreviations).
4. **Adaptive Retrieval**: Implement dynamic k selection based on query intent classification (complex queries $\rightarrow k = 5$; simple queries $\rightarrow k = 1$).
5. **Hallucination Detection**: Add a secondary LLM pass to verify answers against retrieved context and flag unsupported claims.
6. **User Feedback Loop**: Track user ratings/corrections and periodically retrain embeddings; implement RLHF for continuous improvement.
7. **Temporal Awareness**: Add document versioning (e.g., "AY 2025-26") and automatically fall back to previous year's rules if current year is unavailable.
8. **OCR Integration**: Process scanned PDF documents via Tesseract or cloud OCR for full corpus coverage.

Git Repository

The source code for this project is on GitHub. The repository includes all implementation files.

https://github.com/me-in-git/IncomeTax-ChatBot/tree/main

8.1 Screenshots from the App

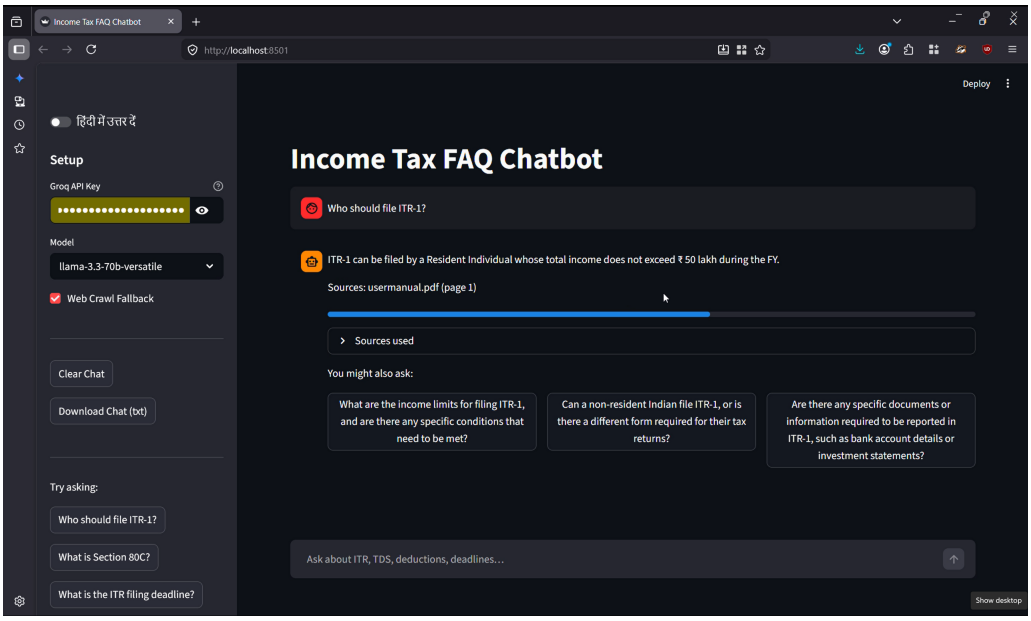


Figure 1: Income Tax ChatBot home screen showing the main chat interface with header, language selector, and input area

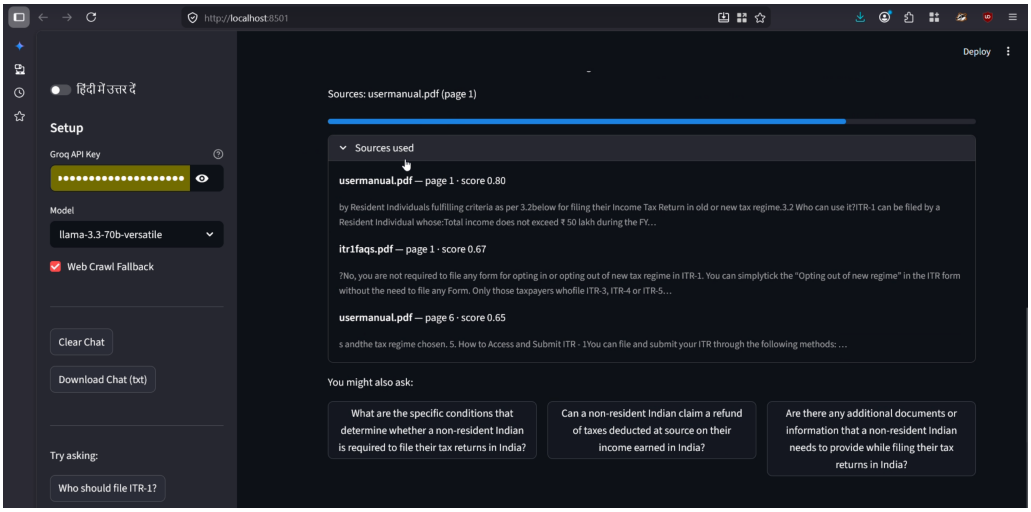


Figure 2: Source attribution feature: Each response includes document name, page number, and confidence score for traceable answers

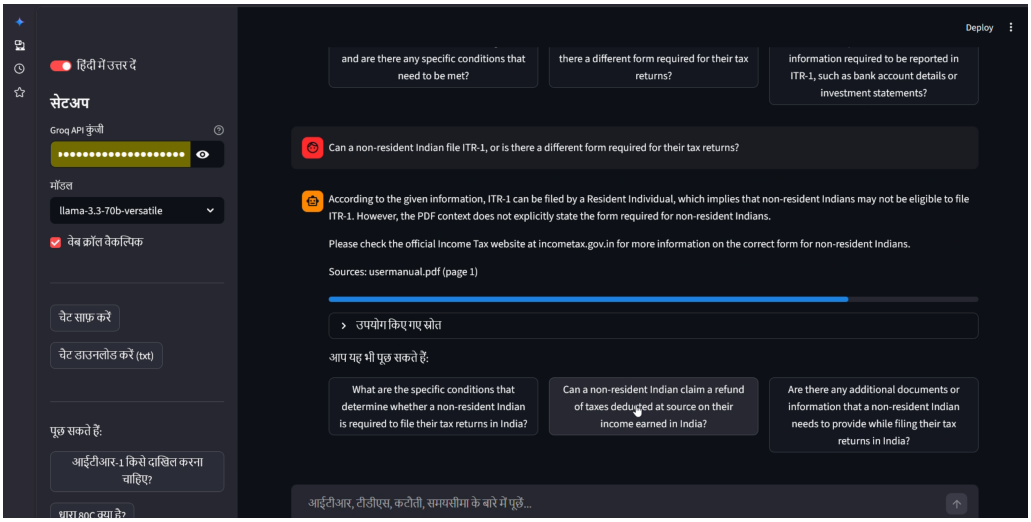


Figure 3: Language toggle button enabling bilingual support (English/Hindi) for Indian taxpayers

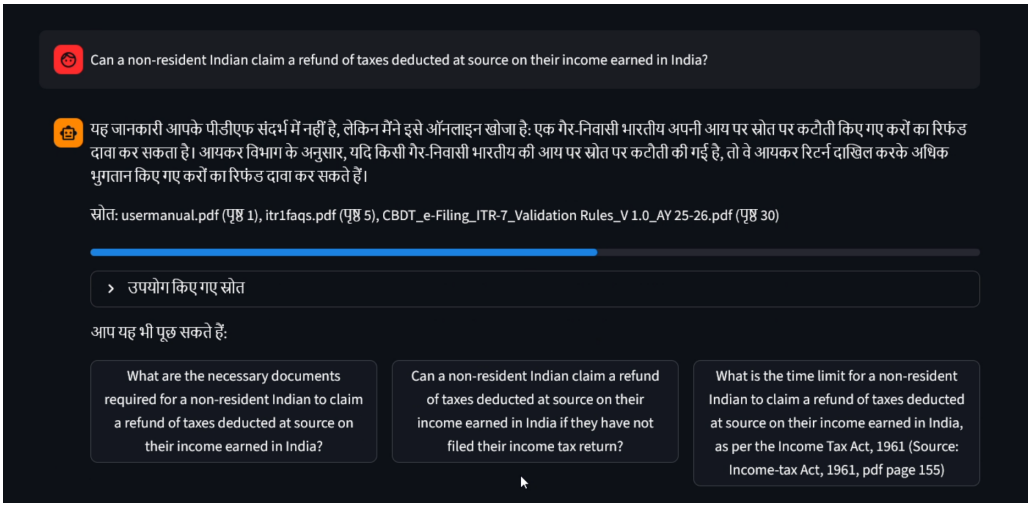


Figure 4: Hindi language response: System answering tax queries in Hindi while maintaining source attribution

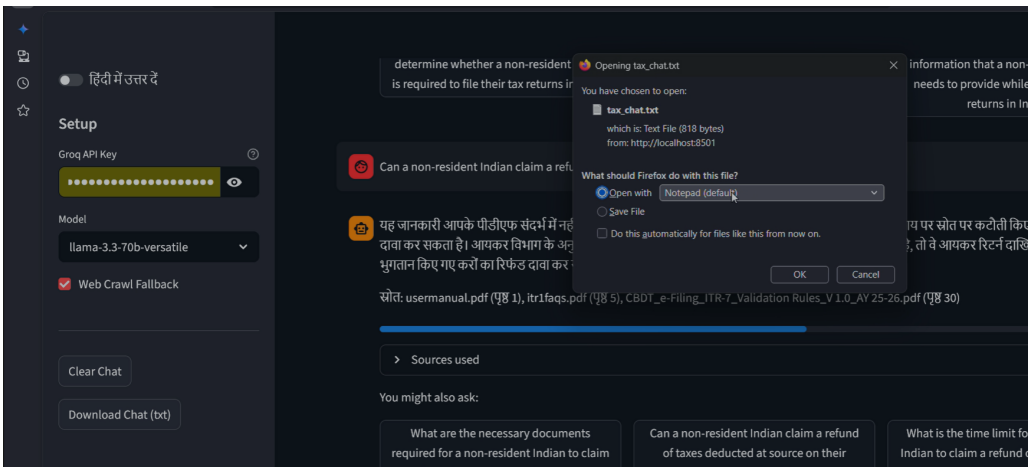
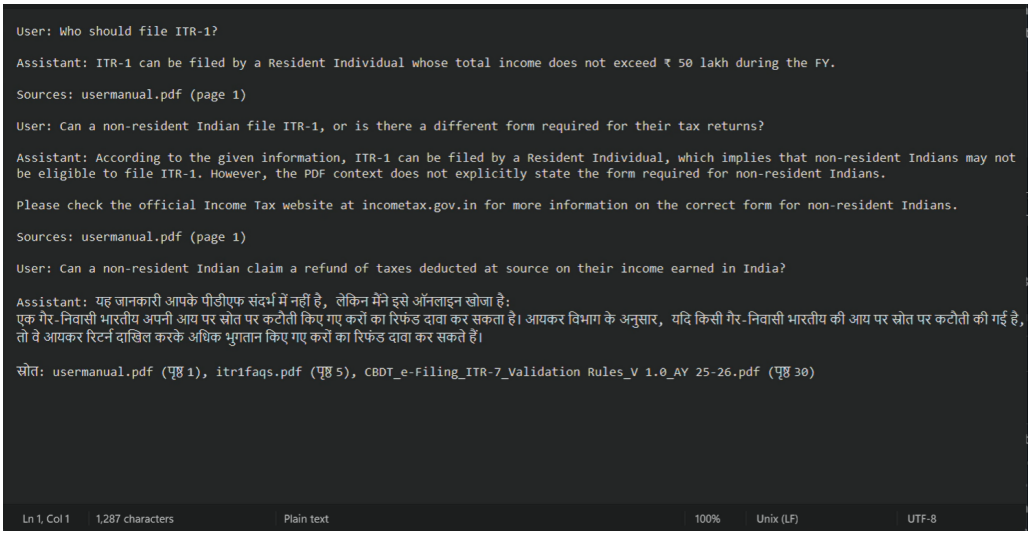


Figure 5: Download conversation feature: Button to export chat history as a text file



User: Who should file ITR-1?

Assistant: ITR-1 can be filed by a Resident Individual whose total income does not exceed ₹ 50 lakh during the FY.

Sources: usermanual.pdf (page 1)

User: Can a non-resident Indian file ITR-1, or is there a different form required for their tax returns?

Assistant: According to the given information, ITR-1 can be filed by a Resident Individual, which implies that non-resident Indians may not be eligible to file ITR-1. However, the PDF context does not explicitly state the form required for non-resident Indians.

Please check the official Income Tax website at incometax.gov.in for more information on the correct form for non-resident Indians.

Sources: usermanual.pdf (page 1)

User: Can a non-resident Indian claim a refund of taxes deducted at source on their income earned in India?

Assistant: यह जानकारी आपके पीडीएफ संदर्भ में नहीं है, लेकिन मैंने इसे ऑनलाइन खोजा है:
एक गैर-निवासी भारतीय अपनी आय पर स्रोत पर कटौती किए गए करों का रिफंड दावा कर सकता है। आयकर विभाग के अनुसार, यदि किसी गैर-निवासी भारतीय की आय पर स्रोत पर कटौती की गई है, तो वे आयकर रिटर्न दाखिल करके अधिक भुगतान किए गए करों का रिफंड दावा कर सकते हैं।

स्रोत: usermanual.pdf (पृष्ठ 1), itr1faqs.pdf (पृष्ठ 5), CBDT_e-Filing_ITR-7_Validation Rules_V 1.0_AY 25-26.pdf (पृष्ठ 30)

Ln 1, Col 1 | 1,287 characters | Plain text | 100% | Unix (LF) | UTF-8

Figure 6: Exported chat history file showing saved conversation with timestamps and source citations

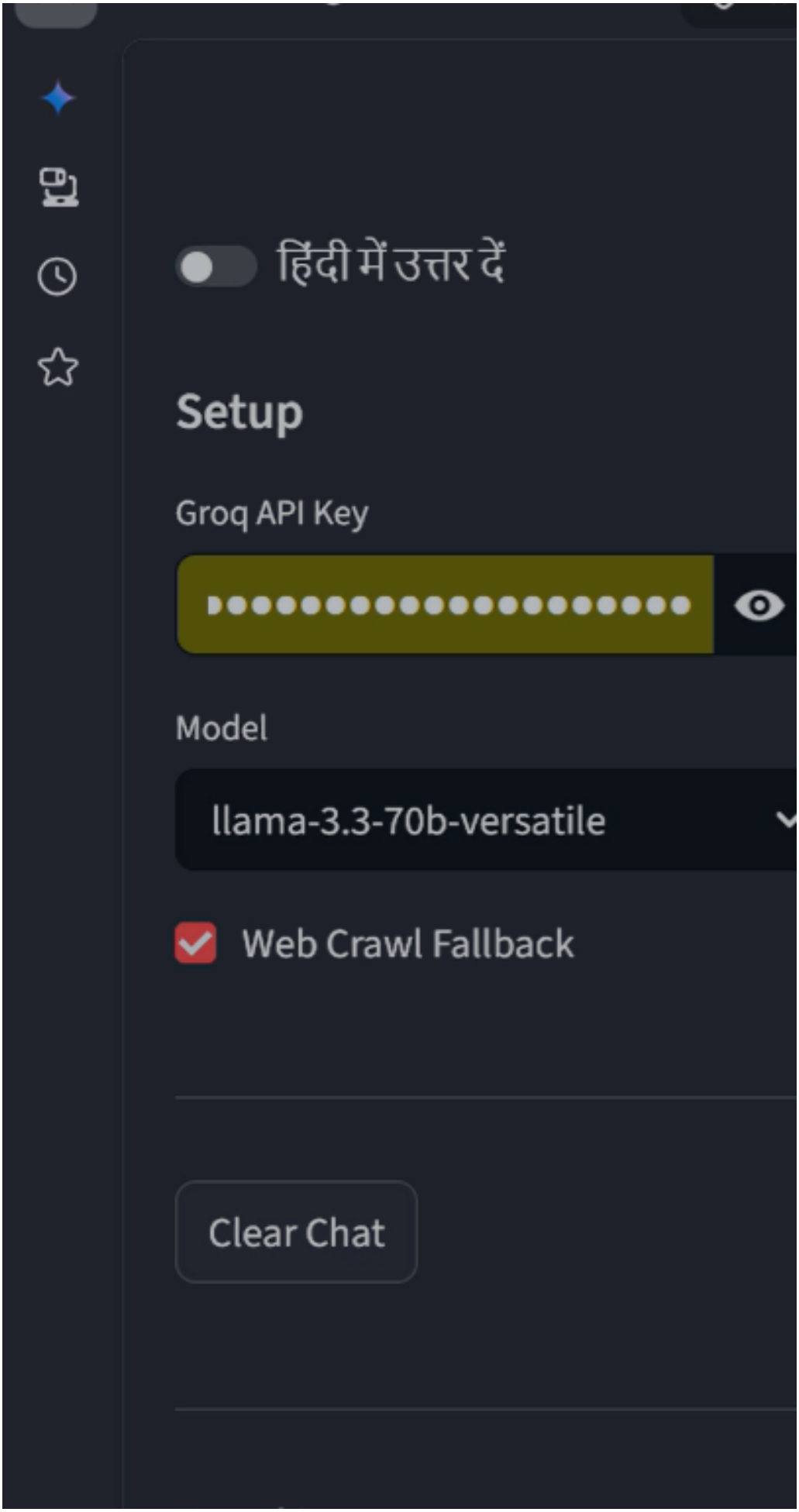


Figure 7: Clear chat button with confirmation dialog to reset conversation history

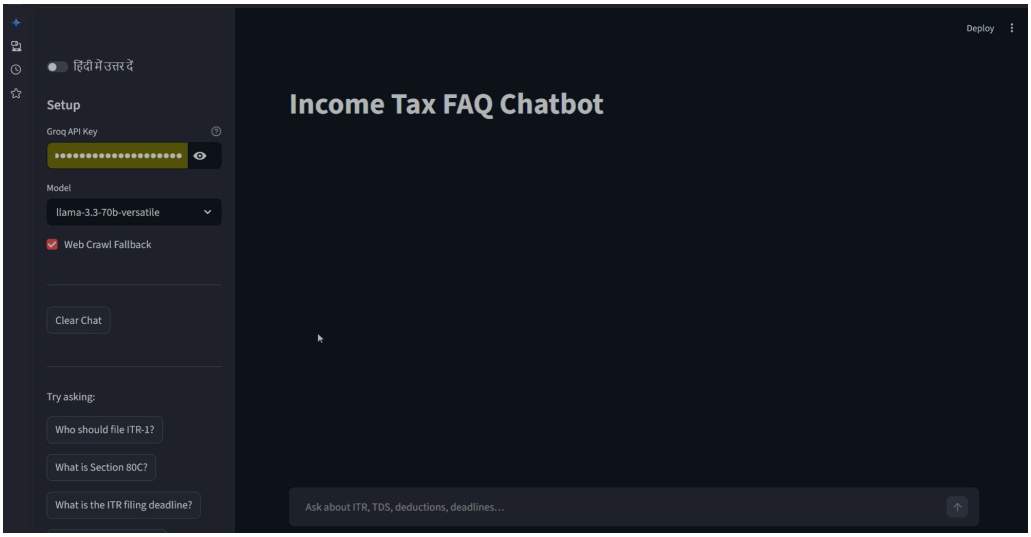


Figure 8: Fresh chat interface after clearing history, ready for new queries

9 Conclusion

The system achieves a balanced design:

- **Accuracy**: Grounds all answers in official PDF documents with source attribution
- **Usability**: Bilingual UI and web fallback for query coverage
- **Maintainability**: Modular architecture separates indexing, retrieval, and generation

Domain-specific RAG systems require careful empirical tuning rather than off-the-shelf defaults.

References

1. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*.
2. Karpukhin, V., Ouz, B., Min, S., et al. (2020). Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*.
3. Nakano, R., Hilton, J., Balaji, S., et al. (2021). WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.
4. Central Board of Direct Taxes (CBDT). (2025). e-Filing ITR Validation Rules, Assessment Year 2025-26. *Government of India*.

Acknowledgements

We used Copilot and Claude for code scaffolding, debugging and report formatting. All analysis was done by us.